Journal of Intelligent & Robotic Systems (2023) 109:26
https://doi.org/10.1007/s10846-023-01951-x

REGULAR PAPER

# Solving Multi-Agent Pickup and Delivery Problems using Multiobjective Optimization

Ana Carolina Queiroz[1] · Alex Vieira[1] · Heder Bernardino[1]

Received: 8 December 2021 / Accepted: 5 August 2023 / Published online: 20 September 2023
© The Author(s), under exclusive licence to Springer Nature B.V. 2023

**Abstract**
Multi-agent pickup and delivery is the problem of allocating tasks for the agents and finding short paths for agents without collisions. These tasks enter the system in different time steps. This article proposes new approaches to this problem based on genetic algorithms in order to optimize the allocation of tasks, minimizing the makespan. We also address this problem as multiobjective, where makespan and service time are minimized. Computational experiments were performed varying the number of agents in a simulated environment of a large-scale warehouse. The results obtained by the proposed approaches were compared with those from the literature and the proposals demonstrated improvements in both objectives.

**Keywords** Multi-agent pickup and delivery · Genetic algorithm · Multiobjective optimization · Real-world application

## 1 Introduction

Due to the high demand for the storage and transport of materials, the need for warehouse automation to increase productivity and reduce costs is increasing largely. In the traditional system, workers pick up the inventory item, take it to a specific place, and prepare it for shipment. As opposed to these workers, one has robotic agents that can perform these tasks faster, better economically, and less subject to human error [1].

Currently, autonomous robots are already used to move around and fetch products. These robots are called here agents, and each agent has to find a path in the warehouse without colliding with each other. This problem is modeled as Multi-Agent Pathfinding (MAPF).

A multi-agent system is composed of several autonomous agents with self-control ability, perceiving their environment and performing actions [2–4]. As affirmed in [3] and reinforced in [5], autonomous agents and their theory are one of the central topics in the field of artificial intelligence (AI). According to [6], multi-agent systems integrate several AI techniques from different fields and offer an efficient alternative for building intelligent systems. Agents are important when solving complex real-world problems, such as those in computer games [7], commerce [8], healthcare [9], and operational research [10]. In this type of system, a set of agents interacts with the environment and with other agents to solve their problems or to conclude some global goals [6]. To successfully interact, agents need to cooperate, coordinate, negotiate, and communicate with each other.

The Multi-Agent Pathfinding (MAPF) [11, 12] is an optimization problem in which one has a set of agents and their respective initial and final positions, and the goal is to find collision-free paths from the agent's initial positions to their final positions. In this problem, the environment is static, remains unchanged, and only the agents act in the environment. The environments, with different sizes and shapes, try to emulate different types of scenarios according to a variety of applications, such as robotics, automated warehouses, and self-driving cars [1, 13–15]. There are variants of these problems being studied in the literature. For instance, Walker

Alex Vieira and Heder Bernardino contributed equally to this work.

✉ Ana Carolina Queiroz
anacarolina@ice.ufjf.br

Alex Vieira
alex.borges@ufjf.edu.br

Heder Bernardino
heder@ice.ufjf.br

[1] Computer Science Department, Universidade Federal de Juiz de Fora, Rua José Lourenço Kelmer, Juiz de Fora 36036-900, Minas Gerais, Brazil

et al. [16] introduced that the agent movement (moving from a cell to another one) takes more than one time step to be completed. Another variant of this problem adds kinematic constraints on the agent's movement [17]. Ferwerda [18] adds multiple waypoints that the agents must visit before reaching their goal.

In this work, the Multi-Agent Pickup and Delivery (MAPD) defined in [19] is considered. In the MAPD problem, the tasks are continuously added to the system and are assigned to the agents. Each task has two positions: pickup and delivery, where the agent first moves to a pickup position and then to the delivery position of a given task. The agent starts the next task as soon as it reaches the delivery position of the task being executed [19]. Differently from the previously presented MAPF problems, the MAPD problem solved here does not take into account the kinematic and spatial characteristics of the agents, the load balancing of the material to be picked up or delivered, battery issues, and possible failures.

Techniques to solve MAPF and MAPD problems can be found in the literature [19–21] and, according to the applications, the number of agents has been increasing to hundreds [1]. It is important to have algorithms that are efficient and require a low computational time, given that MAPF problems (and consequently, MAPD) are NP-hard [22, 23]. As the number of agents increases, the state space grows exponentially [24].

We propose here an efficient approach for solving MAPD problems. For that, a genetic algorithm (GA) [25] is used to allocate the tasks to the agents while the Prioritized Planning method [26] defines the paths of these agents. In order to understand the problem better, analyses of the objectives (makespan and service time) are carried out. As a consequence, we also propose multiobjective optimization approaches for the task allocation part of the problem. The Non-dominated Sorting Genetic Algorithm II (NSGA-II) [27] is adopted for solving the multiobjective optimization problem and a multicriteria decision-making (MCDM) strategy selects a single solution for the path planning step. Computational experiments were performed and the proposed multiobjective approach reached the best results concerning both makespan and service time when compared to other techniques from the literature.

The remainder of this paper is organized as follows. Section 2 presents the related work. Section 3 defines the MAPD problem formally. Section 4 discusses and presents the path-planning method used in this work. Sections 5 and 6 present the GA approaches used for task allocation. Sections 7 and 8 describe and discuss the results of the computational experiments. Finally, Section 9 concludes this paper and describes possible directions for future research.

## 2 Related Work

Some techniques for solving MAPD problems can be found in the literature. In this section, we present these approaches and highlight the main innovations of the proposals when compared to these techniques.

There are two types of MAPD problems: online and offline. In the offline problem [28], the time that a task enters the system is known beforehand. Thus, the algorithm can plan the sequence of tasks to be performed by the agents before the tasks show up, even when the tasks have a release time.

In the online problems studied here, the tasks are only known when they appear in the system [10]. There are different methods to solve MAPD problems that vary in task allocation and path planning. Multi-agent path planning algorithms can be classified into coupled and decoupled [24]. In the first case, a centralized approach defines a path in the joint state space considering all the agents. On the other hand, decoupled techniques compute the plans of each agent separately. Token Passing (TP) and Token Passing with Task Swaps (TPTS) are examples of decoupled algorithms, and Central is a coupled technique. In TP, the free agents select the closest task to execute and use A* twice: from the current position to the pickup position of the task, and from the pickup position to the delivery position. TPTS makes TP more efficient by allowing agents to switch tasks. Central joins available agents to tasks and uses the Conflict-Based Search (CBS) algorithm [24] for path planning. In [19], Central obtained the best results followed by TPTS and TP. However, as TPTS and Central do not allow real-time operations, so only TP was used for experiments in large environments with many agents [19].

Grenouilleau et al. [21] improved the MAPD solution by presenting a decoupled Multi-label algorithm A* (MLA*) algorithm: the agent performs a combined search, calculating the path to the pickup location followed by the path to the delivery location. Along with a new centralized heuristic to plan agents' path to tasks (HBH). The HBH+MLA* algorithm obtained the best results and was compared with the TP.

Queiroz et al. [10] used two algorithms for path planning: Prioritized Planning (decoupled) and CBS (coupled). For the task allocation part, two integer-encoded genetic algorithms were proposed: GA-TA and GA-E. In addition, a heuristic to initialize the population of the genetic algorithms was considered. In GA-TA, both agents and tasks are represented in the candidate solutions. On the other hand, GA-E uses a heuristic to schedule agents to perform tasks and, thus, only a sequence of the agents is represented. Regarding the alloca-

tion of tasks, GA-TA performed better than GA-E. For path planning, CBS obtained the best results, but this approach becomes computationally expensive as the number of agents increases. The results proved the efficiency of task allocation through the use of genetic algorithms, obtaining better results than the Central algorithm, which also uses CBS for path planning.

Two approaches for solving MAPD problems were proposed in [29]. These techniques use Large Neighborhood Search (LNS) for task allocation and plan the paths of the agents using Priority Based Search (PBS). Those methods performed better than the Central algorithm. Additionally, a generalization for MAPD problems, called Multi-Goal (MG-MAPD), is defined. In MG-MAPD, each task can have a sequence of goal locations, which differentiates it from the traditional MAPD where each task has only two goal locations (as considered here). It is important to highlight that makespan is not considered in [29] and here we solve the MAPD problem aiming to minimize the service time and makespan.

Here, we extend the proposals from [10] by developing other genetic algorithm approaches for the task allocation part of MAPD. In addition, we propose a multiobjective formulation for the allocation problem and propose multiobjective approaches for task allocation, with the intention of analyzing and improving the two objectives proposed in the literature.

## 3 Problem Definition

The Multi-Agent Pickup and Delivery problem is specified using an undirected graph $G = (V, E)$, a set of $k$ agents $\{a_1 ... a_k\}$ and a set of $l$ tasks $\{\tau_1 ... \tau_l\}$. Each agent $a_i$ has a start vertex $s_i \in V$, and each task has a start/pickup vertex $s_i \in V$ and a goal/delivery vertex $g_i \in V$ [19].

Time is discretized into timesteps, an agent can move to an adjacent vertex or wait at the current one. The movement from a cell to another one costs one timestep. An agent's path is a sequence of vertex and time pairs $(v, t)$ representing the location of the agent in that timestep. A vertex conflict is represented by a tuple $< a_i, a_j, v, t >$, in which $a_i$ and $a_j$ are agents at the same vertex $v$ at timestep $t$. An edge conflict is represented by $< ai, aj, u, v, t >$, in which $a_i$ and $a_j$ traverse the same edge $(u, v)$ in opposite directions between timesteps $t$ and $t + 1$.

Each agent has a sequence of tasks to perform, and the task is considered concluded when the agent reaches the goal position of the task after it has visited its start position. The tasks enter the system at any time and are already released to be executed.

The agents can be free or busy. Only free agents can be assigned to any task. An agent is free when it is not executing any task or is going to the start position of a task but is not there yet. When the agent arrives at the start position of a task that agent becomes a busy agent and can not change tasks. The agent has to finish executing that task, move to that task's goal position, and then become a free agent that can be assigned to another task.

Every vertex $v$ is an endpoint if it is a start cell of agents or a pickup and delivery cell of a task. There are two types of endpoints: non-task endpoints and task endpoints. Non-task endpoints are all the positions of the park/start location of the agents. Task endpoints are the other locations, where the start and goal locations of the tasks can appear. Agents are allowed to rest (stay indefinitely) in both tasks and non-tasks endpoints.

For an MAPD instance to be considered solvable, it must be considered well-formed. An instance is well-formed if the number of tasks is finite, the number of non-task endpoints is equal to or greater than the number of agents and, for every two endpoints, there is a free path that does not pass through endpoints.

The goal is to conclude the tasks as quickly as possible, and there are two main objectives to be minimized: makespan ($f_{\text{Makespan}}$) and service time ($f_{\text{ServiceTime}}$) . Makespan is the timestep in which the last agent finishes the execution of the last task. This objective function can be defined as

$$f_{\text{Makespan}} = C_{\max} = \max_j \{C_j\},$$

where $C_j$ is the time step when task $j$ is completed, $j = 1, \ldots, l$, and $l$ is the total number of tasks. Service time is the average number of timesteps needed to complete the execution of each task after being added to the system. This objective function can be defined as

$$f_{\text{ServiceTime}} = \frac{1}{l} \sum_{j=1}^{l} (C_j - r_j)$$

where $r_j$ is the release time of task $j$. Makespan and service time are widely used as performance metrics for MAPD.

An example of a solution is illustrated in Fig. 1. In this example, there are 3 agents and 7 tasks ($l = 7$) with the following release times: J1 = 0, J2 = 0, J3 = 1, J4 = 2, J5 = 2, J6 = 3 and J7 = 4. The circles in red in the figure represent the timesteps required for the agents to go from their current position to the pickup location of the tasks. Tasks J2, J5, J4, and J6 were assigned to Agent 1, task J1 was assigned to Agent 2, and tasks J3 and J7 were assigned to Agent 3. The execution of a given task is initiated when the agent arrives at this task pickup position. The initial location of Agent 1 is also the pickup location of task J2. Agent 1 finishes J2 and takes 1 timestep to arrive at the pickup location of J5. The delivery location of J5 is the same as the pickup location

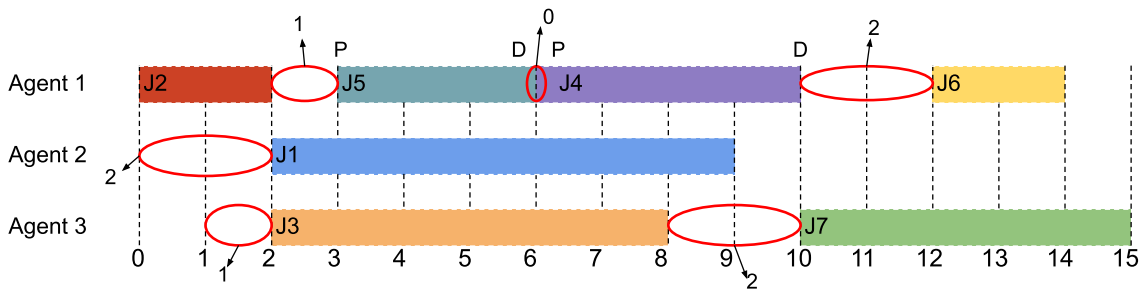


**Fig. 1** Gantt chart of a solution example. The horizontal bars represent the timesteps the agent is executing a task, the red circles represent the timesteps required for the agents to go from their current position to the pickup location of the tasks

of J4 and it takes 2 timesteps from the delivery of J4 to the next pickup, J6. The makespan in this example is 15, and the service time is $(9 + 2 + 7 + 8 + 4 + 11 + 11) \div 7 = 7.42$.

Considering that the problem has two goals, it can be treated as a multiobjective optimization problem. A multiobjective optimization occurs when there is more than one conflicting objective to be optimized. This work deals with two optimization problems: mono-objective and multiobjective. The first is when only one objective is optimized, and, given the importance of makespan in the literature, its optimization is considered a priority in this case. Both service time and makespan objectives are optimized when multiple objectives are considered. Additional details are presented in Section 6.

## 4 Path Planning

As previously commented, multi-agent path planning approaches can be decoupled and coupled. Conflict-Based Search (CBS) [24] is an example of a technique able to find optimal solutions for this problem. In a coupled approach, when the size of the problem (the number of agents) increases, it also increases computationally exponentially, in contrast with the decoupled approaches that run relatively fast, but optimality and even completeness are not always guaranteed [24, 30].

In the decoupled situation, each agent plans its path separately. The agent searches for its destination knowing the current status of all other agents. The decoupled approaches increase efficiency but at the expense of sub-optimal solutions. In this work, a decoupled approach is used, since the coupled algorithms do not scale to larger instances. The algorithm used is the Prioritized Planning described in [10, 26] that uses Cooperative A* [31].

The Prioritized Planning algorithm creates a list of priorities to determine which agent has the highest priority in executing the path planning. The agents with the largest estimated distances to arrive at their goal position have the highest priority. Thus, it is expected that the solution does not increase the makespan since the agents with the longest path are executing first with a small number of collisions [26].

Prioritized Planning uses Cooperative A* that creates a reservation table that saves the path of each agent. This reservation table has the pair location and time for all the agents. This way each agent checks the table to verify if the position is free to move.

Classic A* is used to find the shortest path for each agent. It is a graph traversal and path search algorithm often used to find the lowest cost path between two points on a known map. It works by maintaining a list of "open" vertexes, which initially contain only the initial vertex. At each iteration, A* determines a vertex to extend, forming a path. Vertex $v$ with the smallest $f(v)$ is selected, where $f$ the sum of the cost of the path with an estimate of the cost required to extend the path to the goal (defined by a heuristic function). The selected vertex $v$ is removed and extended. Extending means that all neighboring vertex are visited and if they have not been added previously, these vertexes are added to the list of "open". A* terminates when the destination node is reached, or there is no eligible vertex to be extended.

### 4.1 Collision Avoidance

To find a path to the pickup location and then to the delivery location, two A* calls are employed: one from the agent's current location to the pickup location, and another from the pickup to the delivery location. Unlike classic A*, in which

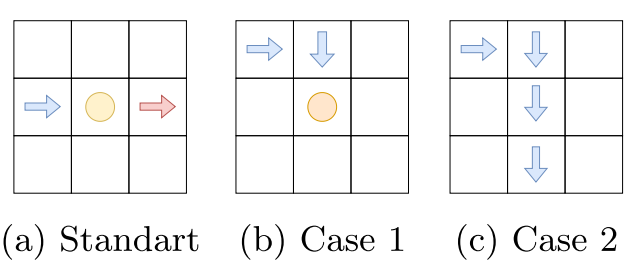


**Fig. 2** Representation of the map with the path planning of the agents. The arrows are the agent's movement and the circles are the agents' final positions

the problem is concluded when the final position is reached, here it is still necessary to allocate the agent somewhere when its final position is reached. As discussed in [21], the A* algorithms find a path for the agents and it is assumed that they stay in that position until the end of the timesteps. So, in the A* search algorithm, when an agent checks if it can go to the final position (represented in Fig. 2a by the yellow circle), one of the following situations may occur: (i) another agent has a pickup or delivery node in that same position (Fig. 2b), or (ii) another agent is scheduled to pass through that same position (Fig. 2c). These both cases block the agent in (a) to arrive at its final position, as other agents will pass by the same position in the future and A* considers that the agent in (a) will rest indefinitely at that location. In addition, the agents in (b) and (c) may not plan to pass by the final position of the agent in (a) as A* assumes that position as occupied.

Another strategy is to make the agents return to their park location [10, 28]. In this case, the agent does not block any cell that is a task-endpoint (pickup and delivery positions), but this strategy is expensive as it calculates the path to the park location unnecessarily when there are many tasks to solve. And when there are a small number of tasks it is not feasible for the agents to keep returning to their park location as there may be free positions closer than the park location.

When using A*, one is not interested if the final position is a pickup or delivery node. The final position is passed to the algorithm and it is assumed that the agent stays in that position until the end of the time horizon, even when the final position is a pickup location.

Different from that approach, Grenouilleau et al. [21] are interested if the final position is a pickup or delivery, it uses an algorithm called Multi-label A* algorithm (MLA*) in which they perform a search for their pickup location and to the delivery location in a combined search, resulting in a path to the agent‘s location to the pickup, and then to delivery location, avoiding the situation in Fig. 2. In this work, until the agent arrives at the pickup location of a task, it can be assigned to another task, which usually happens when a large number of tasks are being released in each timestep. This results in the Multi-label algorithm A* (MLA*) computing unnecessary path plans.

### 4.2 The Adopted Path Planning Approach

Algorithm 1 presents the outline of the algorithm used in this work (Prioritized Planning). The algorithm receives a list of agents to plan the path and each agent has a final position $p$. It is considered that the agent only has to arrive at the final position (line 10). Algorithm 2 presents the modified A* used, when the agent arrives at their goal location then the agent goes to the closest free endpoint as a park location (rest indefinitely), which may coincide to be the final position of the agent or not (line 4 and 5). In a situation where there is no free endpoint reachable (line 5), the A* search continues (line 8) until the agent arrives at its goal location and finds a location to stay indefinitely in its current state of the search tree.

**Algorithm 1** Pseudo-code of the path planning algorithm.

```
1: Sort the list of agents according to the estimated distance from the
   agent's position to the final position, from farther to closer.
2: for all agents a in the ordered list do
3:     for all agents k in the list of agents do
4:         if agent k != agent a then
5:             if agent k is blocking the position p of agent a then
6:                 Move agent k to the closest free endpoint
7:             end if
8:         end if
9:     end for
10:    Get a path from A* (a, p) (Algorithm 2)
11: end for
12: Return the solution found
```

**Algorithm 2** Modified A*.

```
1: Create an initial node from the initial location, add the node to Open;
2: while Open is not empty do
3:     Get the node with the smallest f from Open
4:     if node n == goal location then
5:         if agent a can go from their goal location to a free endpoint
   then
6:             Return the path to goal location + path from goal to free
   endpoint
7:         else
8:             Continue
9:         end if
10:    end if
11:    Expand node n to the adjacent node, if the location is not blocked,
   add this node to Open
12: end while
```

The agents can only stay indefinitely in both tasks and non-tasks endpoint positions. Also, the instance is well-formed and, consequently, there is a park position for every agent and a path between endpoints that does not pass through any other endpoint. That way, an agent is not parked in a place where other agents and obstacles block it. As more tasks are added to the system, that previously free position can now be a task position and the agent in that position may block another one responsible for handling that task.

To minimize the problem of having an agent blocking the agent's final position, in Algorithm 1 it is checked every time a task is assigned to an agent if there is any agent blocking the path. A blocking agent is requested to move to a near-free endpoint (line 6).

## 5 Task Allocation with Genetic Algorithms

The task allocation part specifies which tasks are assigned to the agent and in which order the agent should execute them. A

**Algorithm 3** Pseudo-code of the proposed procedures using GA.

1: $timestep = 0$
2: **while** the stop criterion is not met **do**
3: $list_of_agents_to_plan = \emptyset$
4: **if** a task has entered the system **then**
5: Add task to the set of tasks
6: Get the sequence of tasks for each agent using a genetic algorithm
7: **end if**
8: **if** an agent has finished executing a task **then**
9: Set agent as free
10: Set the final position of an agent as the pickup location of its next task
11: Add agent to the list_of_agents_to_plan
12: **end if**
13: **if** an agent has arrived at the pickup location of a task **then**
14: Set agent as busy
15: Set final position of the agent as the delivery location of the task
16: Remove task from the set of tasks
17: Add agent to the list_of_agents_to_plan
18: **end if**
19: Call the path planning algorithm (Algorithm 1)
20: $timestep = timestep + 1$
21: **end while**

genetic algorithm (GA) [25] is used in this work to optimize the sequence of tasks that each agent is going to solve [10].

Algorithm 3 presents the outline of the algorithm described in this work. In every timestep, it checks if any new task has entered the system. The task allocation algorithm is executed whenever a new task enters the system. In line 6 given the new set of tasks, the genetic algorithm returns the solution, a sequence of tasks for each agent. Then in line 8, the algorithm checks if the agents have finished executing a task. If so, the agent is set to free, and set the goal position of the agent is set as the pickup position of the next task in the task sequence. Then it checks if the agent has arrived at the pickup position of the task it is assigned, if so, sets the agent as busy, and the goal position of the agent is set to the delivery location of this task. In line 19 we sort the list of agents in descending order of the distances of the agent position to their goal position. For each agent in the list, we execute the path planning Algorithm 1.

The proposed GA is encoded using two strings: a string of tasks and a string of agents. In each position of the chromosome, an agent is associated with a task, generating a sequence of tasks for each agent. Thus, one can obtain a sequence of tasks for each agent. Figure 3 shows a representation of a solution with 10 tasks and 5 agents. In this illustrative example, Agent 3 executes task 7 and then task 5; Agent 2 executes task 1, followed by tasks 4 and 6. The other agents and tasks follow the same process.

The main objective of the GA is to improve solutions, generating better task sequences for each agent, the GA does this by improving the fitness values of the population using different operators like mutation, crossover, and selection. The selection of the parents to create a new population is performed by a tournament selection.

The crossover and mutation operators are different and applied separately on each string (agents and tasks). This difference occurs as tasks cannot be duplicated but agents can, i.e., an agent can handle several tasks but a task can only be handled by one agent.

The crossover operator is designed in the two strings, a One Point crossover in the string of agents, where one point in the string is chosen and the remaining agents are swapped as shown in Fig. 4. For the string of tasks, we use the Order One crossover permutation (OX), shown in Fig. 5. In this crossover, two cut points are selected, and the middle substring of the first parent is copied to the child. These copied values are marked in the second parent to not be copied again. The remaining genes are copied from Parent 2 to the empty genes in the child in the same order they appear.

The mutation can occur in the chromosome of tasks or in the chromosome of agents. Figure 6 shows the mutation in the string of tasks, that is the pairwise interchange neighborhood mutation (IM). Two tasks are chosen from the chromosome and their positions are exchanged. In the chromosome of the agents, the mutation randomly chooses a position in the chromosome and the current value ($v_c$) is changed to another integer value in $\{v_c - \delta \times k, \ldots, v_c + \delta \times k\}$, where $k$ is the number of agents. Also, the new value ($v_n$) is fixed when this is not in the correct range, from 1 to $k$: $v_n$ is modified to $k + v_n$, when $v_n < 1$, and $v_n - k$, when $v_n > k$. The mutation of the chromosome of the agents is illustrated in Fig. 7. The

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Tasks | 7 | 1 | 3 | 9 | 5 | 4 | 8 | 10 | 2 | 6 |
| Agents | 3 | 2 | 0 | 4 | 3 | 2 | 4 | 1 | 4 | 2 |

**Fig. 3** Representation of a candidate solution

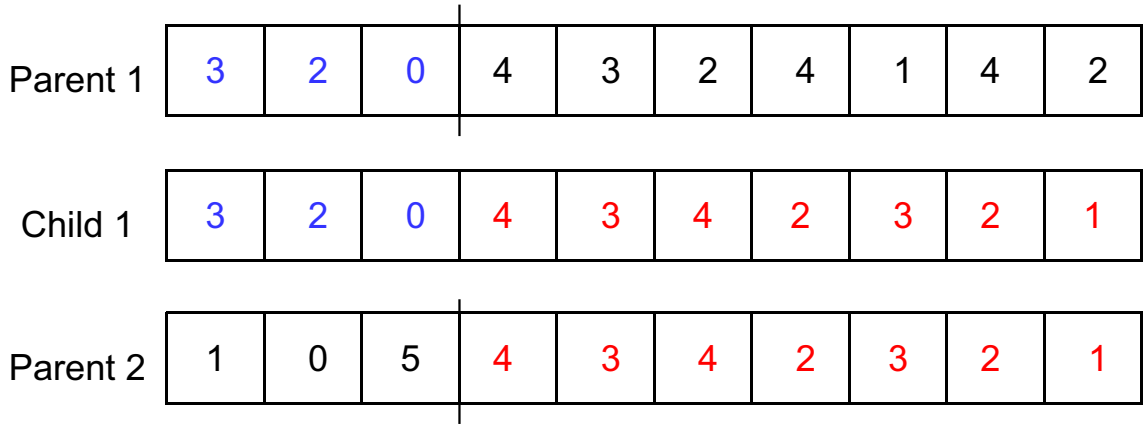


**Fig. 4** Example of crossover (one point) in the strings of agents

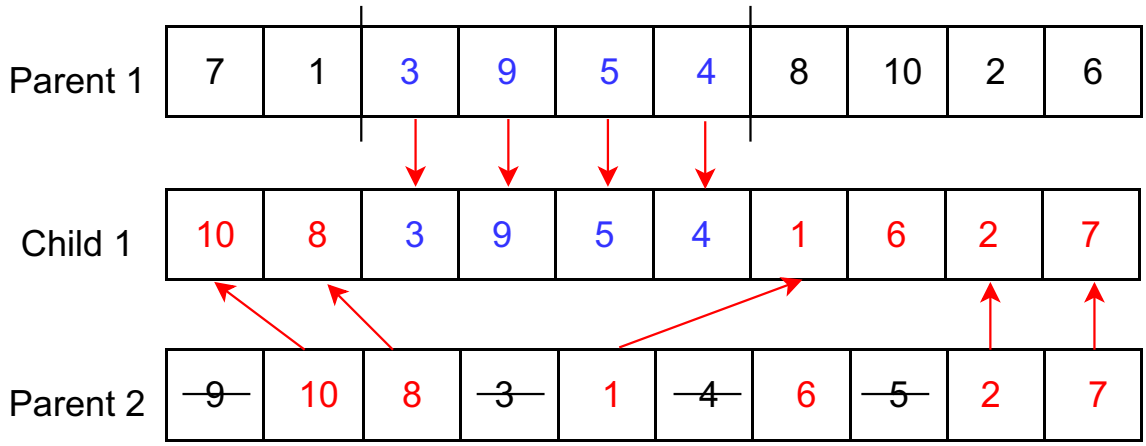


**Fig. 5** Example of crossover (order one) in the strings of tasks

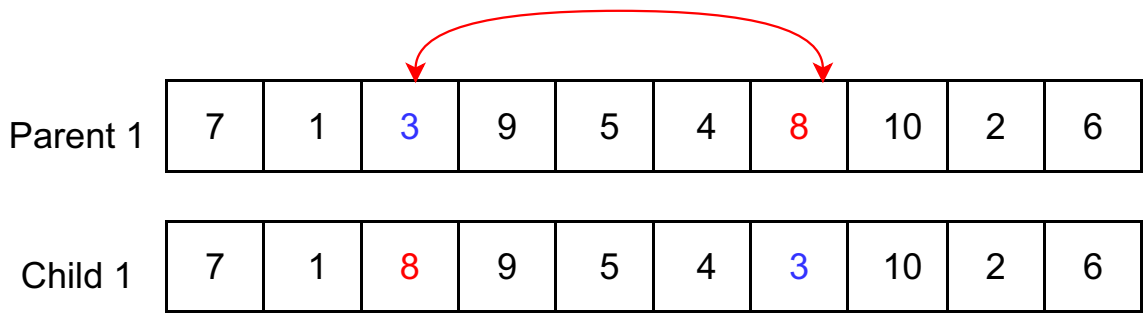


**Fig. 6** Example of mutation (the pairwise interchange neighborhood mutation -IM) in the strings of tasks

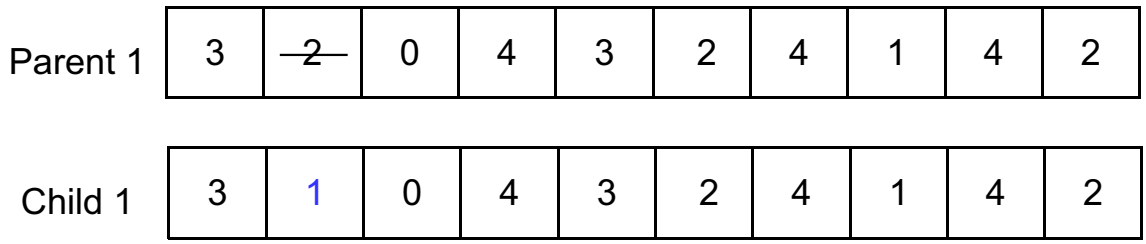


**Fig. 7** Example of mutation in the strings of agents

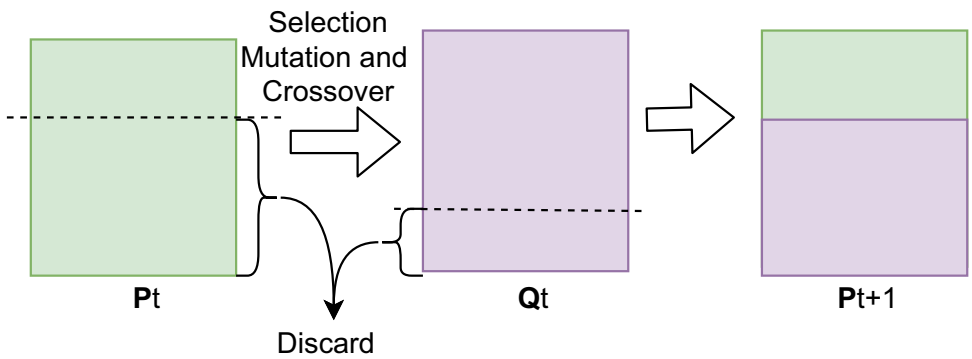


**Fig. 8** Replacement procedure of GA-TA

string of agents works as a circular list in the case of values drawn greater or less than the total number of agents.

Each individual has a string of agents with their sequence of tasks to be solved, the fitness is calculated by approximating the values of the path planning. As mentioned in Section 4, path planning with multi-agents is very costly with the collision avoidance calculations, so an approximation of the values that would be obtained by path planning is necessary. The approximation is the number of timesteps required to move from one point to the other considering the shortest path and without considering possible collisions and changes of route. When the genetic algorithm finishes then the real path planning is performed (line 19 in Algorithm 3). Algorithm 1 is used as a strategy to plan the path, now considering collisions, changing the values of the optimal makespan and service time.

### 5.1 Mono-Objective -GA-TA

GA-TA is the generational genetic algorithm described in [10]. Figure 8 shows the structure of the algorithm. The population $P_t$ is the current population, and the mutation and crossover described in Section 5 are used to create a new population $Q_t$ generated from $P_t$.

Through selection made by elitism, the worst individuals from the two populations $P_t$ and $Q_t$ are discarded, and the best individuals from both populations are copied to a new population $P_{t+1}$ keeping the same population size as $P_t$. In the next generation, the current population became $P_t$ and the search process continues until the stop criterion is reached.

### 5.2 Initialization of the Population

An improvement was proposed in [10] to help the search process of the genetic algorithms: a greedy heuristic initializes the population. One individual is generated with the greedy heuristic and the remainder of the population is generated randomly.

The heuristic chooses for the agents a sequence with tasks that are closer to them, using an approximate value for the distance. The heuristic uses a distance matrix between the position of the agents in that given time to the pickup position

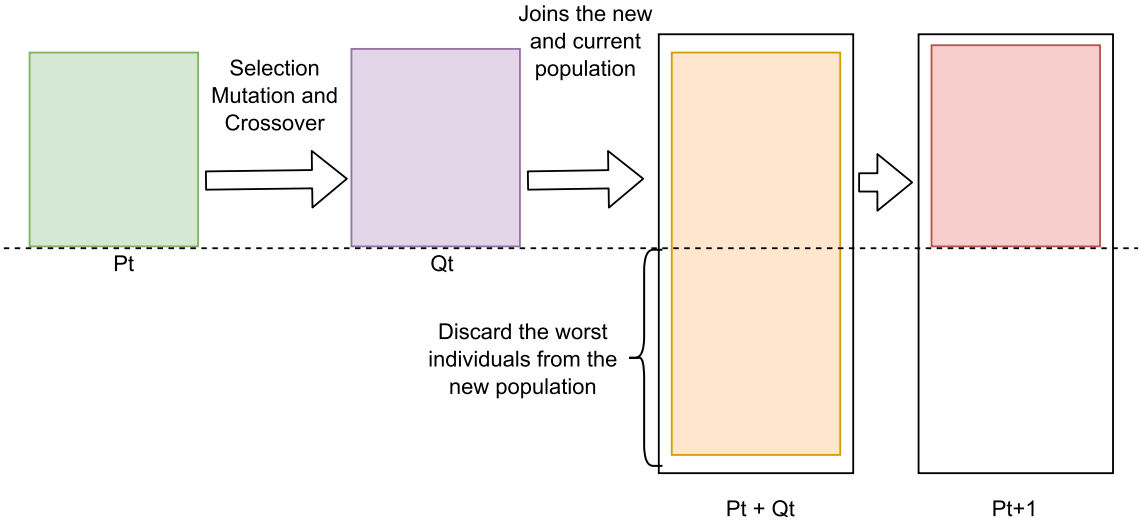


**Fig. 9** Replacement procedure of Greedy

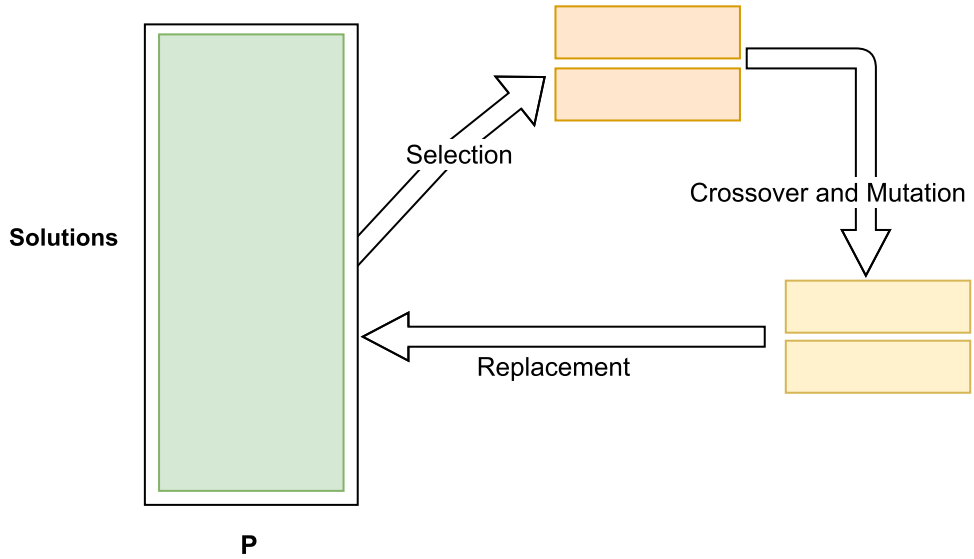


**Fig. 10** Replacement procedure of Steady State

of the tasks. The agent and task with the lowest distance value are added to the chromosome and the task is removed from the matrix. The agent position is set to the delivery point of the task chosen, and the agent has a new time which is the distance between the agent position to task pickup plus the task pickup location to the task delivery location. In each run, the matrix is updated as the distance between the agent and all the tasks changes with the new position of the agent. In [10] computational experiments demonstrated the effectiveness in reducing the makespan values of the solutions, using this heuristic compared to using the entire population randomly.

## 6 Proposed Approaches

As stated in Section 5, the GA is used to optimize task assignment for agents. We propose here using GA approaches with the following replacement strategies for allocating the tasks to agents in online MAPD problems: a generational Greedy strategy and a steady state procedure. In addition, we propose a new problem formulation for MAPD that uses multiple objectives, namely, NSGA-M, NSGA-ST, and NSGA-O.

In this case, a multiobjective GA is adopted. In both, mono-objective and multiobjective situations, the difference

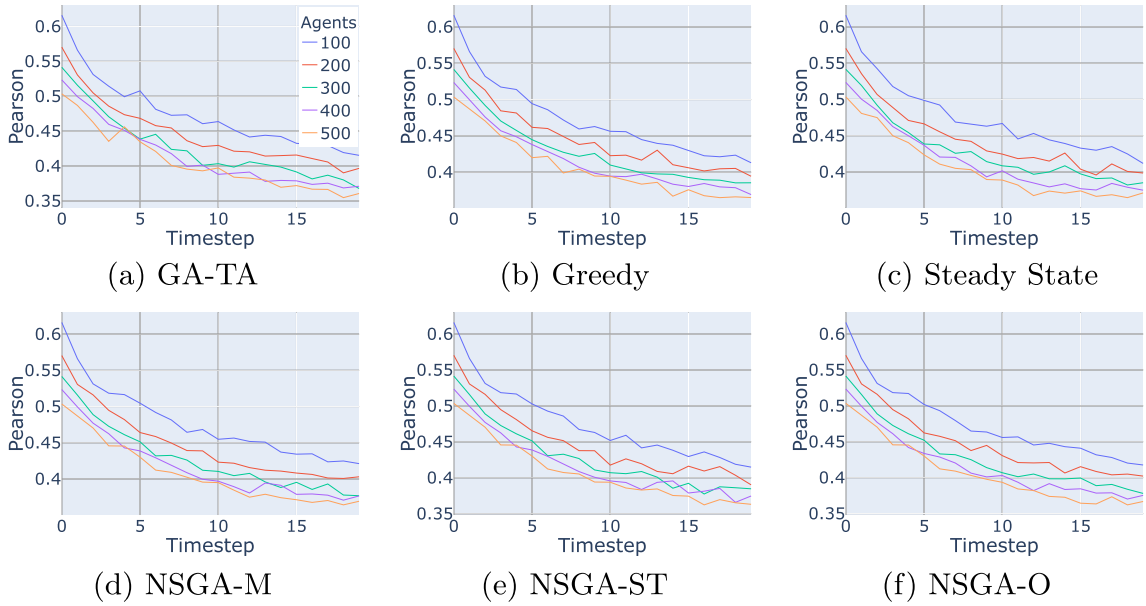


**Fig. 11** Correlation between the two objectives: makespan and service time. The y-axis is the correlation coefficient and the x-axis is the number of timesteps each time the genetic algorithm is called

among the techniques is the replacement strategy. More details regarding the proposed methods are presented in the following sections.

### 6.1 Mono-Objective - Greedy

Greedy is a generational genetic algorithm. Figure 9 represents a diagram with the structure of the algorithm. The current population is $P_t$, and the mutation and crossover in individuals of $P_t$ create a new population $Q_t$ with the same size as $P_t$. To create the new population $P_{t+1}$ the population $P_t$ and $Q_t$ are united, and the fewer fit individuals are discarded according to the makespan objective, keeping the population size of $P_t$.

### 6.2 Mono-Objective - Steady State

The other genetic algorithms are known as generational. In this algorithm, we use a steady-state strategy. In the generational approach, the entire population gets replaced each generation, while in the steady-state approach only a few individuals in the population are replaced each generation. The tournament selection does not replace the selected individuals in the population, instead, the children of the selected parents are added to the population, then the two worst individuals are removed from the population so the population size remains constant. Unlike generational genetic algorithms, steady-state approaches do not evaluate all the candidates in the entire population. Our approach differs in the tournament process, where two individuals compete: (i) a candidate solution in the sequence, and (ii) another individual randomly chosen. The best of them is selected. For the decision-making, the lowest fitness is chosen according to makespan. The process is demonstrated in Fig. 10.

### 6.3 Multiobjective - NSGA-II

An analysis of the objectives was carried out here to evaluate whether improving one objective is not improving the other. Pearson's correlation coefficient was used to check the existing relationship between the pair of objectives: makespan and service time. The higher the absolute value the greater the correlation between the goals. Data were achieved by 1000 randomly generated individuals and 30 independent runs in each algorithm.

It can be seen in Fig. 11 that as timesteps increase and more tasks accumulate and enter the system, the correlation between objectives decreases. The greater the number of agents, the lower the correlation between the objectives. Results were also generated using the Spearman correlation; the results obtained are similar and are available in the supplementary material.

The small correlation values between makespan and service time indicate that this optimization problem can be treated as a multiobjective problem. This is similar to the procedure presented in [32], where an analysis of multiple objectives was performed, and conflicting objectives were considered when the correlation is negative or close to zero.

So we propose here a multiobjective formulation for MAPD. Also, a multiobjective optimization approach is applied to this problem.

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) is a multiobjective genetic algorithm. In NSGA-II the individuals are selected according to non-dominated-sorting or the Pareto ordering and crowding distance [27].

The Pareto Ordering consists of classifying a population $P$ in $d$ subsets such that $F_1 = \{$ all non-dominated individuals from $P$ and who are not dominated by any other of $P\}$ (rank 1), $F_2 = \{$all non-dominated individuals from $P \setminus F_1$ and who are not dominated by any other from $P \setminus F_1\}$(rank 2), ... $F_d = \{$all non-dominated individuals of $P \setminus \{F_1 \cup F_2 \cup ... \cup F_{d-1}\}$ and that are not dominated by any other of $P \setminus \{F_1 \cup F_2 \cup ... \cup F_{d-1}\}\}$ (rank $d$), where $P = F_1 \cup F_2 \cup ... \cup F_d$.

Figure 12 demonstrates the difference between dominated solutions to non-dominated solutions and their fronts. The individuals in the first front (F1) are not dominated by any other individual of the population and dominate over the others. The same happens with the individuals in rank 2 (F2), who dominates all the other individuals, except in relation to the individuals in the first front.

The crowding distance of each individual is based on the distance it is from neighboring individuals, for each objective function within the same rank. The crowding distance is the semi-perimeter formed by the cuboid, for example, illustrated in Fig. 12, and is a measure of how close an individual is to its neighbors. The higher the value of the crowding dis-

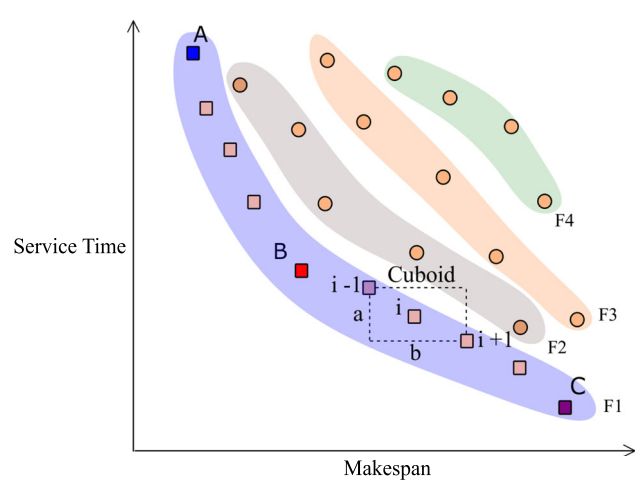


**Fig. 12** Example of dominated and non-dominated solutions and each front from the population and calculating the individual's crowding distance $i_k$: $CD(i_k) = a + b$

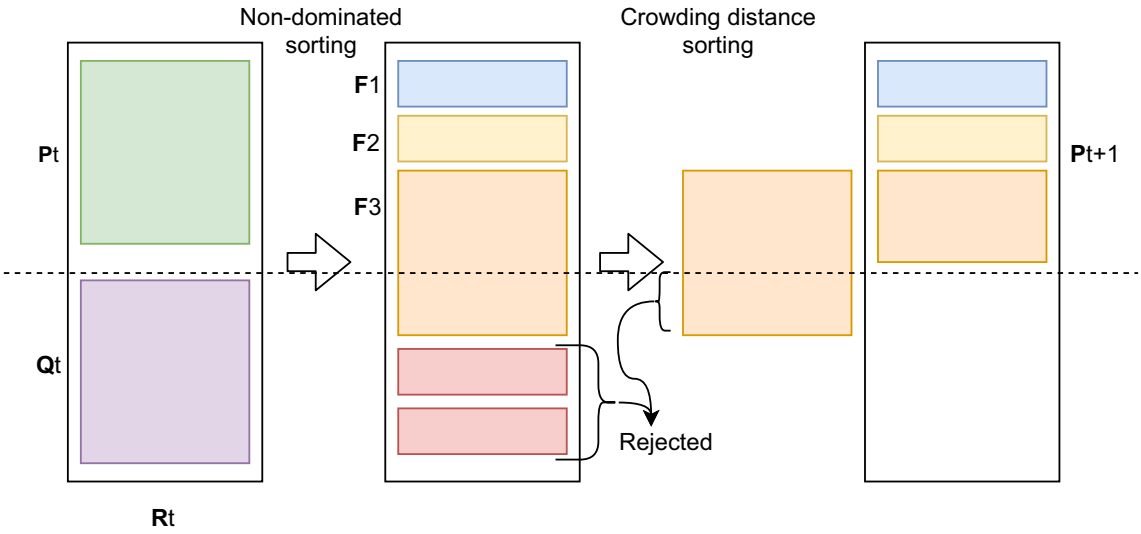


**Fig. 13** Replacement procedure of NSGA-II (adapted from [33])

tance, the further away the solution is from its neighbors in the space of objectives, making their preference greater in selection for the next generation of the population.

The new population is selected using the best individuals according to their rank. Individuals that have lower rank continue in the population and individuals with a bigger rank, that are more dominated, are discarded. If two individuals have the same rank, the crowding distance is used to untie and the individual with the biggest crowding distance is chosen. An illustration of the population replacement of NSGA-II is presented in Fig. 13.

Three approaches using NSGA-II are proposed here. Similar to other metaheuristics, a population of solutions is obtained by NSGA-II. In addition, a non-dominated set of solutions can be extracted. These solutions are equally good and a decision-making step is required to select the most appropriate to the problem being solved. The search is performed with respect to the multiple objectives in these three proposed approaches. However, at the end of the search, there is a multi-criteria decision-making procedure that selects a single solution from the set of candidate solutions of the final population. Thus, the three proposed approaches differ in the selection of the individual to be executed.

In **NSGA-M**, the decision is choosing the best individual according to Objective 1, that is, the individual with the lowest Makespan. According to this strategy, Individual A is selected in Fig. 12.

In **NSGA-ST**, the decision is selecting the best solution according to Objective 2, that is, that individual with the lowest Service time. In this case, Individual C is selected in Fig. 12.

**NSGA-O** chooses the individual that has the best results for both objectives. The best result is obtained by normalizing the results for both objectives as their orders of magnitude are different, assigning each objective to an axis and then calculating the distance of each individual to the origin of the axis, choosing the one that is closest to the origin. Individual B is selected in Fig. 12.

## 7 Computational Experiments

Experiments were done with the main objective to analyze the different approaches of genetic algorithms and discuss the problem using a multiobjective algorithm. The instances used are from [19]. The experiments are performed in a large environment with 81x81 grid cells shown in Fig. 14. Five instances (five maps) are used, varying in the number of

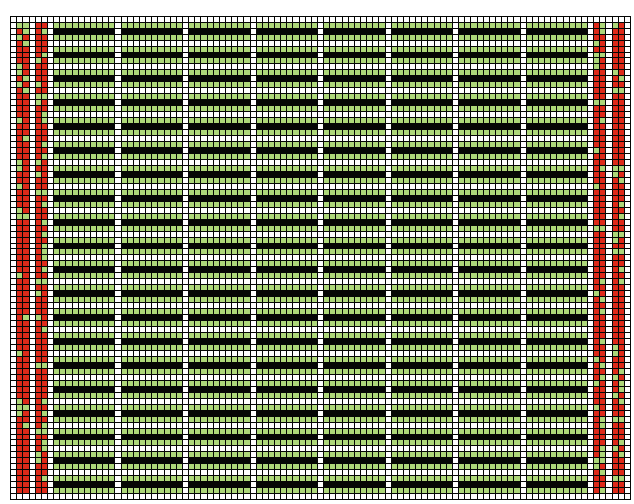

**Fig. 14** Large environment with 500 agents (adapted from [19]). Black cells are blocked positions, green cells are task endpoints, and red cells are non-task endpoints

**Table 1** Average of the makespan results

| Ag | GA-TA | Greedy | SS | NSGA-M | NSGA-ST | NSGA-O |
|---|---|---|---|---|---|---|
| 100 | 771.60⋆ | 773.73 | 769.87⋆ | **767.07** | 769.47⋆ | 769.63⋆ |
| 200 | 460.07 | 450.73 | 455.20 | **444.30** | 447.10⋆ | 445.37⋆ |
| 300 | 359,03 | 352.93 | 357.43 | 346.77⋆ | **345.63** | 348.40⋆ |
| 400 | 304.37 | 301.83 | 303.77 | 300.33⋆ | 299.53⋆ | **297.47** |
| 500 | 285.03 | 275.87⋆ | 278.57 | 273.10⋆ | 276.20⋆ | **272.67** |

"Ag" is the number of agents and SS is the Steady State algorithm
The best results for each instance are highlighted

agents with 100, 200, 300, 400, and 500 agents. The total amount of tasks is 1000 and the rate used is 50 tasks per timestep, meaning that it takes 20 timesteps for the last tasks to enter the system, and then no more tasks are added. The code is written in C++ and the source codes and instances used are available.[1]

The proposed approaches (five techniques) and GA-TA are applied to the MAPD problems and 30 independent runs are performed for each method in each problem. The size of the population in the genetic algorithms is set to 20 and the number of generations is 50.

The parameters used in the genetic algorithms for the creation of the new population by tournament selection is 1%. 80% was used as the crossover value, 50% for the mutation, and 30% for the $\delta$. In the GA-TA the value of the selection by elitism is 40%. In the Steady State algorithm, in order to maintain the same number of function evaluations, the number of generations used is 500.

To evaluate the proposals, the methods are compared to TP [19] and HBH + MLA* [21], both solving the MAPD instances with the same environment and the number of agents. The methods developed in this work and TP were implemented. The results obtained by HBH + MLA* are those presented in [21].

## 8 Results and Analysis

The results are shown in Tables 1 and 2. Table 1 presents the results regarding the average makespan and Table 2 shows the results of the average service time obtained by the proposed approaches and GA-TA.

Comparing the proposed Greedy and Steady State techniques with GA-TA [10], the two proposals obtained better results according to both cases, makespan and service time, with the different numbers of agents tested (100, 200, 300, 400, and 500). These three approaches solve the mono-objective version of the problem and their difference is the strategy used to replace the population. Thus, the replacement procedures used here performed better than that from the literature.

Among the tested algorithms, one can see the advantage of the NSGA algorithms in comparison to the others, the 3 algorithms of this type presented better results, evaluating both the service time and the makespan. The results of each algorithm show little variation among themselves, remaining between 1% and 5% of difference between the best and worst cases, without a clear distribution of this variation concerning the number of agents.

To compare the performances of the algorithms, they are also tested for statistical significance using Wilcoxon with the null hypothesis that the distribution of the samples has equal medians. The results are considered statistically different here when the p-value is smaller than 0.05. A ⋆ indicates that these values are statistically similar to the results of the best algorithm, represented in bold (same distribution). All p-values are provided in the supplementary material.

Comparing the results of the makespan in Table 1, one has that the results of all NSGA-II algorithms are statistically similar, almost all the results of the other algorithms show statistically significant differences, except the GA-TA and Steady State algorithms in tests with 100 agents, and the Greedy algorithm in the case of 500 agents. And comparing the results of the service time in Table 2 the NSGA-II algorithms are mostly statistically similar, except for the NSGA-M in the case of 300 agents, where it showed a significant difference. The other algorithms showed statistically significant results in their entirety.

The best proposal is selected and compared to other approaches from the literature. Initially, we ranked the techniques with respect to the sum of the normalized makespan and service time values. The normalized values are calculated using $\frac{\text{value - minimum_value}}{\text{maximum_value} - \text{minimum_value}}$, where the value is normalized using both the minimum and maximum values for each problem and objective. Concerning the sum of the normalized makespans and service times, the proposed NSGA-M obtained the best result (0.7175), followed by NSGA-O (0.8521) and NSGA-ST (1.1836). The sums of the normalized results of NSGA-M and NSGA-O are similar, but the results found by NSGA-M are not the best ones or statistically similar to the best ones in all tested cases. Thus,

[1] https://github.com/carolladeira/MAPD-NSGA.

**Table 2** Average of the service time results

| Ag | GA-TA | Greedy | SS | NSGA-M | NSGA-ST | NSGA-O |
|---|---|---|---|---|---|---|
| 100 | 354.51 | 351.86 | 352.84 | 350.27* | **350.17** | 350.32* |
| 200 | 200.98 | 199.15 | 199.65 | **198.67** | 198.69* | 198.79* |
| 300 | 153.06 | 151.20 | 151.55 | 150.65 | **150.39** | 150.55* |
| 400 | 130.03 | 128.53 | 129.09 | 128.00* | **127.89** | 127.99* |
| 500 | 118.62 | 116.50 | 116.96 | 116.00* | 116.11* | **115.97** |

"Ag" is the number of agents and SS is the Steady State algorithm
The best results for each instance are highlighted

we consider that NSGA-O is the best-performing proposal. Table 3 contains the results found by NSGA-O and those achieved by two other methods from the literature.

When compared with the algorithms, TP and HBH+MLA*, the NSGA-O showed significantly better results than the others in both compared objectives. From the results shown, it is possible to observe a considerable improvement in the performance of the NSGA-O with the increase in the number of agents, highlighting the results of makespan with 500 agents being 45.57% better compared to HBH+MLA*, showing its efficiency and scalability.

Figure 15 presents boxplots of the makespan, service time, and run time reached by the algorithms proposed here, and Fig. 16 those found by NSGA-O, TP, and HBH+MLA*. In the literature, the run time is presented in ms/timestep. In order to compare the techniques, we converted the values to seconds. To improve visualization, boxplots of the run times of NSGA-O and HBH+MLA* are available in the supplementary material.

One can observe that the NSGA-II methods have improvements regarding both makespan and service time, they do not increase the computational cost compared to TP and the other proposed genetic algorithm approaches. Among NSGA-II algorithms there is no substantial difference between them in computational cost as shown in the boxplots.

Compared to HBH+MLA* this approach has the difference of allowing the agent to switch tasks while the agent doesn't reach the pickup position of the task, which causes the algorithm to reprocess the agent's path. Therefore we achieved a better makespan and service time, at the expense of processing. Even though one can't make a direct comparison because of the differences in the processors.

**Table 3** Comparison of results with TP and HBH+MLA*, where MS is makespan and ST is service time

| Ag | TP | | HBH+MLA* | | NSGA-O | |
|---|---|---|---|---|---|---|
| | MS | ST | MS | ST | MS | ST |
| 100 | 1018 | 463.25 | 800 | 363 | **769.63** | **350.32** |
| 200 | 745 | 330.18 | 606 | 208 | **445.37** | **198.79** |
| 300 | 746 | 301.96 | 507 | 157 | **348.40** | **150.55** |
| 400 | 788 | 289.07 | 491 | 136 | **297.47** | **127.99** |
| 500 | 946 | 284.24 | 501 | 125 | **272.67** | **115.97** |

The best results for each instance are highlighted

## 9 Concluding Remarks and Future Work

The Multi-Agent Pickup and Delivery Problems were studied in a large environment, tasks enter the system at different times, and each task has a pickup and delivery position. This problem consists of assigning tasks to the agents and finding the agent's path without collisions.

Different approaches of genetic algorithms were proposed to improve the allocation of the tasks Greedy and Steady State. An analysis was made of the relationships between the two main objectives: makespan and service time, and after this analysis, we also propose a multiobjective optimization using 3 algorithms: NSGA-M, NSGA-ST, and NSGA-O.

The algorithms proposed in this work were compared with GA-TA, another technique from the literature. The tests showed that the algorithms can solve problems with an increasing amount of agents in a large environment, showing their scalability. By comparing with the mono-objective approach, the results also attested to the success of the multiobjective approach.

NSGA-O was compared with other techniques from the literature and was the algorithm with the best overall results in terms of makespan and service time. Also, the results indicated an increasing margin of efficiency when increasing the number of agents. Regarding the processing time, HBH+MLA* obtained the best values. In addition, the run times of both HBH+MLA* and the proposed NSGA-O are in general much better than those achieved by TP. However, it is important to highlight that the applicability of the methods depends on their processing times, the length of time the real agents take to execute the tasks, and the time interval in which new tasks are inserted into the environment. Also, it is worth exploring the potential for improving runtime using high-performance computing. In particular, it is easy to parallel the proposal as a population of candidate solutions is evaluated in each iteration. The tests were not executed on the same machine and it may be necessary to run them in

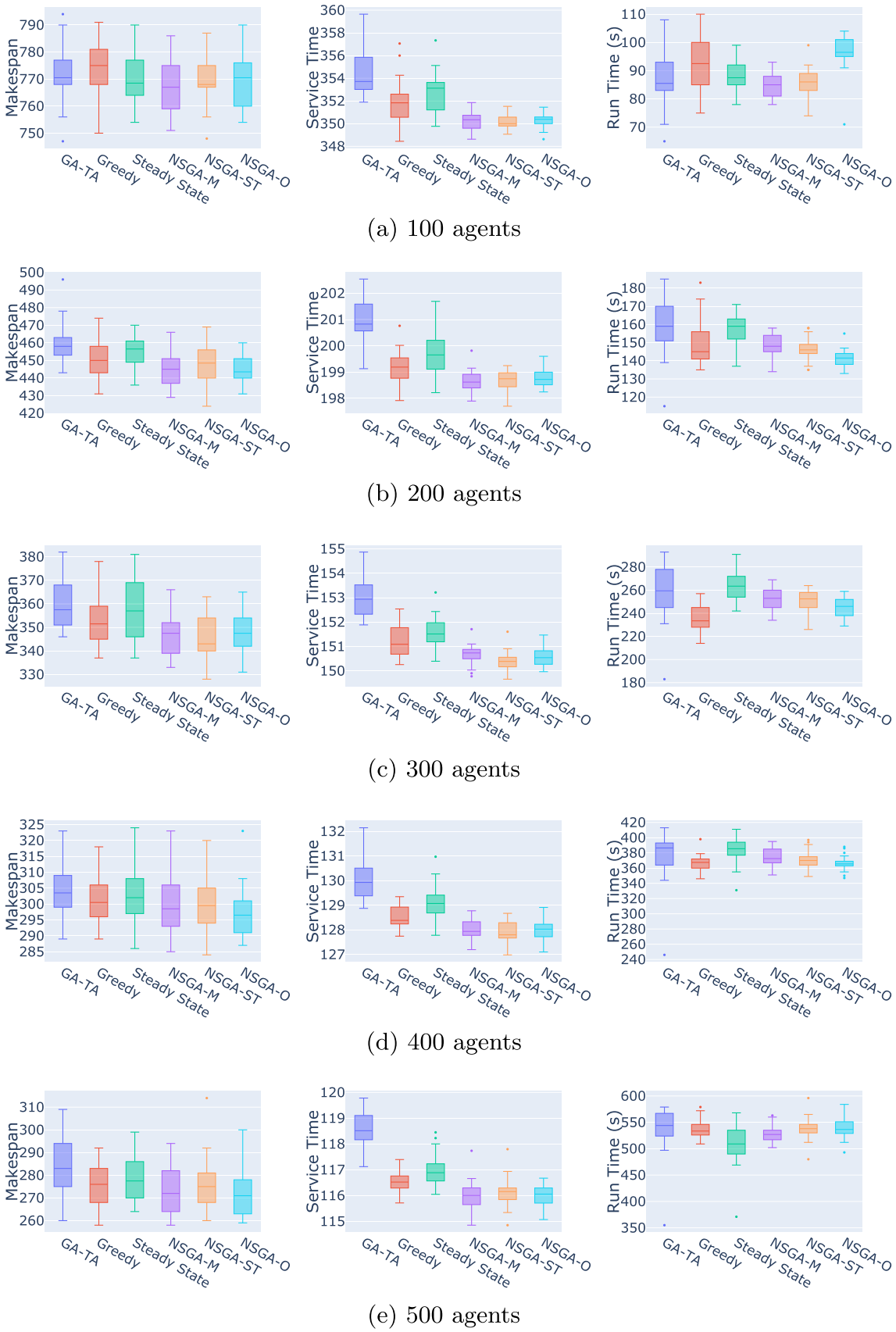


**Fig. 15** Boxplots of the proposed algorithms: GA-TA, Greedy, Steady State, NSGA-M, NSGA-ST, NSGA-O

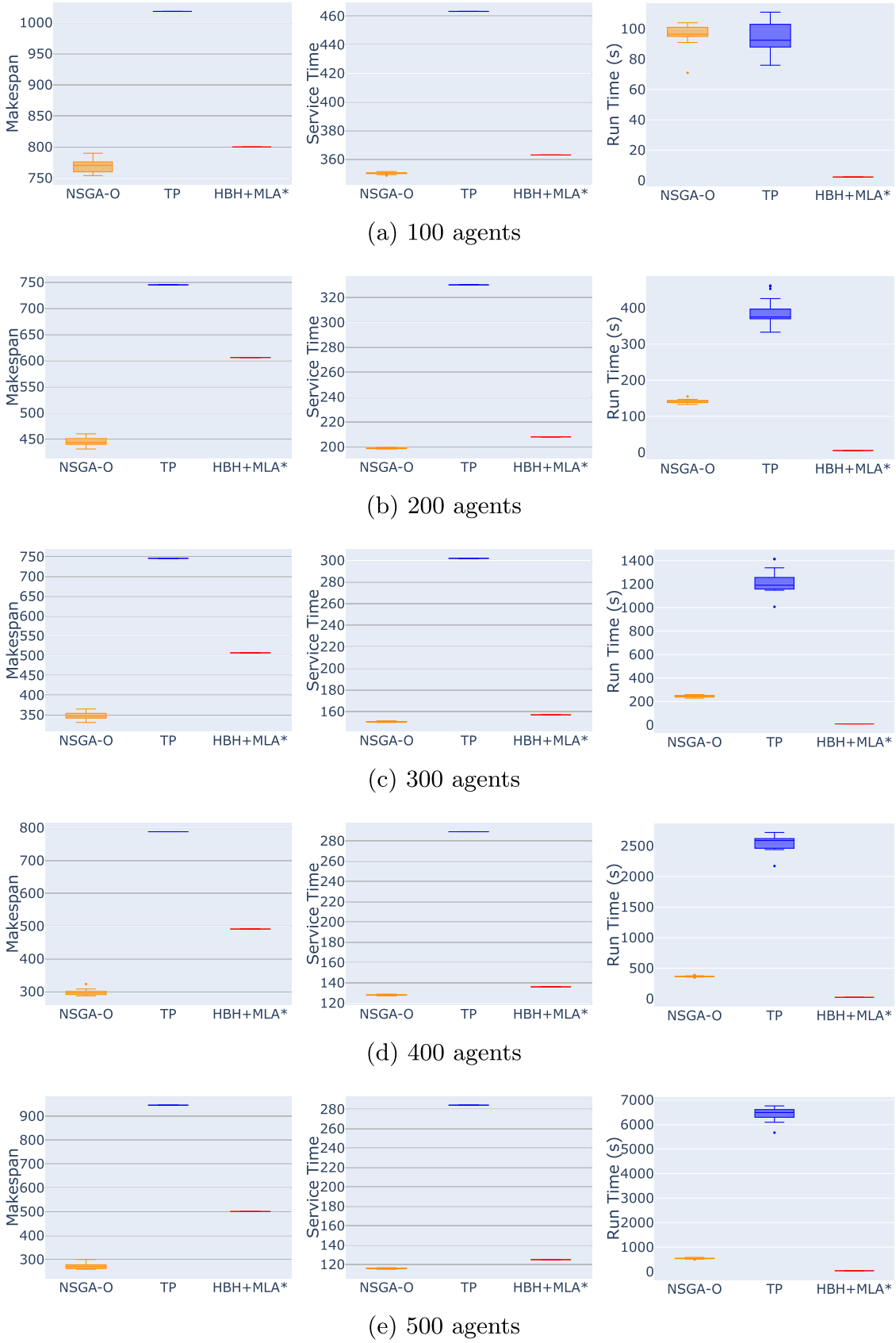


**Fig. 16** Boxplots of NSGA-O, TP, and HBH+MLA*

the same environment in the future for a better comparison in terms of processing time.

For future research, we intend to carry out tests considering task constraints, such as load capacity and time limits. We also intend to evaluate other forms of approximating the makespan and service time values used in the genetic algorithm. In addition, we plan to use these techniques in different and more realistic scenarios. It is also intended to include other heuristics to initialize populations of genetic algorithms. Furthermore, we have in view preventing the exchanging of tasks of free agents.

**Acknowledgements** The authors thank the financial support provided by CNPq, Capes, FAPEMIG and FAPESP.

**Author Contributions** All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Ana Carolina Ladeira Costa Queiroz, Alex Borges Vieira and Heder Soares Bernardino. The first draft of the manuscript was written by Ana Carolina Ladeira Costa Queiroz and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

**Funding** This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, FAPEMIG (APQ-00337-18 and CEX APQ-02955/1) and CNPq (312682/2018-2 and 311206/2018-2).

**Code or data availability** All code and data associated with the current submission is available at https://github.com/carolladeira/MAPD-NSGA.

## Declarations

**Competing Interests** The authors have no relevant financial or non-financial interests to disclose

**Ethics approval** Not applicable

**Consent to participate** Not applicable

**Consent to publish** Not applicable

## References

1. Wurman, P.R., D'Andrea, R., Mountz, M.: Coordinating hundreds of cooperative, autonomous vehicles in warehouses. AI Mag. **29**(1), 9–20 (2008)
2. Franklin, S., Graesser, A.C.: Is it an agent, or just a program?: A taxonomy for autonomous agents. In: Intelligent Agents III, Agent Theories, Architectures, and Languages, ECAI '96 Workshop (ATAL), August 12–13, Proceedings. Lecture Notes in Computer Science, vol. 1193, pp. 21–35. Springer, Budapest, Hungary (1996)
3. Jennings, N.R., Sycara, K., Wooldridge, M.: A roadmap of agent research and development. Autonomous Agents and Multi-Agent Systems **1**, 7–38 (1998)
4. Wooldridge, M.J.: An Introduction to MultiAgent Systems. John Wiley & Sons Inc, USA (2002)
5. Petrović, V.M., Kovačević, B.D.: Avilab - gamified virtual educational tool for introduction to agent theory fundamentals. Electron. **11**(3), (2022)
6. Oprea, M.: Applications of multi-agent systems. In: Reis, R. (ed.) Information Technology, pp. 239–270. Springer, Boston, MA (2004)
7. Van Lent, M., Laird, J., Buckman, J., Hartford, J., Houchard, S., Steinkraus, K., Tedrake, R.: Intelligent agents in computer games. In: AAAI/IAAI, pp. 929–930 (1999)
8. Sandholm, T.: emediator: A next generation electronic commerce server. Comput. Intell. **18**(4), 656–676 (2002)
9. Shakshuki, E., Reid, M.: Multi-agent system applications in healthcare: Current technology and future roadmap. Proced. Comput. Sci. 52, 252–261 (2015). The 6th International Conference on Ambient Systems, Networks and Technologies (ANT-2015), the 5th International Conference on Sustainable Energy Information Technology (SEIT-2015)
10. Queiroz, A.C.L., Bernardino, H.S., Vieira, A.B., Barbosa, H.J.: Solving multi-agent pickup and delivery problems using a genetic algorithm. In: Brazilian Conference on Intelligent Systems, pp. 140–153. Springer, (2020)
11. Stern, R., Sturtevant, N.R., Felner, A., Koenig, S., Ma, H., Walker, T.T., Li, J., Atzmon, D., Cohen, L., Kumar, T.K.S., Barták, R., Boyarski, E.: Multi-agent path finding: Definitions, variants, and benchmarks. In: Proc. of the 12th Intl. Symposium on Combinatorial Search, SOCS, Napa, California, pp. 151–159 (2019)
12. Felner, A., Stern, R., Shimony, S.E., Boyarski, E., Goldenberg, M., Sharon, G., Sturtevant, N.R., Wagner, G., Surynek, P.: Search-based optimal solvers for the multi-agent path finding problem: Summary and challenges. In: Proc. of the 10th Intl. Symposium on Combinatorial Search, SOCS 16-17, USA, pp. 29–37. AAAI Press, Pittsburgh, PA (2017)
13. Veloso, M.M., Biswas, J., Coltin, B., Rosenthal, S.: Cobots: Robust symbiotic autonomous mobile service robots. In: Proc. of the 24th Intl. Joint Conference on Artificial Intelligence, IJCAI 2015, July 25-31, 2015, p. 4423. AAAI Press, Buenos Aires, Argentina (2015)
14. Morris, R., Pasareanu, C.S., Luckow, K.S., Malik, W., Ma, H., Kumar, T.K.S., Koenig, S.: Planning, scheduling and monitoring for airport surface operations. In: Planning for Hybrid Systems, Papers from the 2016 AAAI Workshop, February 13. AAAI Workshops, vol. WS-16-12, pp. 608–614. AAAI Press, Phoenix, Arizona, USA (2016)
15. Ma, H., Koenig, S., Ayanian, N., Cohen, L., Hönig, W., Kumar, T.K.S., Uras, T., Xu, H., Tovey, C.A., Sharon, G.: Overview: Generalizations of multi-agent path finding to real-world scenarios. CoRR abs/1702.05515, 1–4 (2017)
16. Walker, T.T., Sturtevant, N.R., Felner, A.: Extended increasing cost tree search for non-unit cost domains. In: Proc. of the 27th Intl. Joint Conference on Artificial Intelligence, IJCAI, Stockholm, Sweden, pp. 534–540 (2018)
17. Hönig, W., Kumar, T.K.S., Cohen, L., Ma, H., Xu, H., Ayanian, N., Koenig, S.: Summary: Multi-agent pathfinding with kinematic constraints. In: Proc. of the 26th Intl. Joint Conf. on Artificial Intelligence, IJCAI 2017, August 19-25, 2017, pp. 4869–4873. ijcai.org, Melbourne, Australia (2017)
18. Ferwerda, A.: Extending the multi-label a* algorithm for multi-agent path finding with multiple waypoints. Technical report, Delft University of Technology (2020)
19. Ma, H., Li, J., Kumar, T.K.S., Koenig, S.: Lifelong multi-agent path finding for online pickup and delivery tasks. In: Proc. of the 16th Conf. on Autonomous Agents and MultiAgent Systems. AAMAS '17, pp. 837– 845. Intl. Foundation for Autonomous Agents and Multiagent Systems, Richland, SC (2017)
20. Lam, E., Bodic, P.L., Harabor, D.D., Stuckey, P.J.: Branch-and-cut-andprice for multi-agent pathfinding. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelli-

gence, IJCAI, Macao, China, August 10-16, 2019, pp. 1289–1296 (2019)
21. Grenouilleau, F., van Hoeve, W.-J., Hooker, J.N.: A multi-label a* algorithm for multi-agent pathfinding. In: Proceedings of the International Conference on Automated Planning and Scheduling, vol. 29, pp. 181–185 (2019)
22. Yu, J., LaValle, S.M.: Structure and intractability of optimal multirobot path planning on graphs. In: Twenty-Seventh AAAI Conference on Artificial Intelligence (2013)
23. Nebel, B.: On the computational complexity of multi-agent pathfinding on directed graphs. In: Proceedings of the Thirtieth International Conference on Automated Planning and Scheduling, Nancy, France, October 26-30, pp. 212–216 (2020)
24. Sharon, G., Stern, R., Felner, A., Sturtevant, N.R.: Conict-based search for optimal multi-agent pathfinding. Art Intell **219**, 40–66 (2015)
25. Holland, J.H.: Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. MIT Press, Cambridge, MA, USA (1992)
26. Liu, M., Ma, H., Li, J., Koenig, S.: Task and path planning for multiagent pickup and delivery. In: Proc. of the 18th Intl. Conf. on Autonomous Agents and MultiAgent Systems, AAMAS '19, May 13-17, 2019, pp. 1152– 1160. Intl. Foundation for Autonomous Agents and Multiagent Systems, Montreal, QC, Canada (2019)
27. Deb, K., Agrawal, S., Pratap, A., Meyarivan, T.: A fast elitist nondominated sorting genetic algorithm for multi-objective optimization:Nsga-ii. In: International Conference on Parallel Problem Solving from Nature, pp. 849–858. Springer, (2000)
28. Liu, M., Ma, H., Li, J., Koenig, S.: Task and path planning for multiagent pickup and delivery. In: Proc. of the 18th Intl. Conf. on Autonomous Agents and MultiAgent Systems, pp. 1152–1160. Intl. Foundation for Autonomous Agents and Multiagent Systems, (2019)
29. Xu, Q., Li, J., Koenig, S., Ma, H.: Multi-goal multi-agent pickup and delivery. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 9964–9971. IEEE, (2022)
30. Ionescu, V., van der Meer, M., van Kooten, B., Paardekooper, G., Teunissen, J.: A guide to solving pathfinding problems with multiple agents. Technical report, Delft University of Technology (2020)
31. Silver, D.: Cooperative pathfinding. AIIDE **1**, 117–122 (2005)
32. Pereira, A.A.S., Barbosa, H.J.C., Bernardino, H.S.: Predator-prey techniques for solving multiobjective scheduling problems for unrelated parallel machines. In: EMO. Lecture Notes in Computer Science, vol. 10173, pp. 484–498. Springer, Münster, Germany (2017)
33. Deb, K.: Multi-objective Optimization Using Evolutionary Algorithms. Wiley-Interscience series in systems and optimization. John Wiley & Sons, USA (2001)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

**Ana Carolina Queiroz** received his master's degree (2022) and bachelor's degree (2017) in Computer Science from the Federal University of Juiz de Fora (UFJF), Brazil. Her research interests include evolutionary computation, optimization techniques and software engineering.

**Alex Borges Vieira** received the Ph.D. degree in computer science from Universidade Federal de Minas Gerais, Brazil. He is currently an Associate Professor with the Computer Science Department, Universidade Federal de Juiz de Fora. During 2019, he has been a Visiting Professor with the National Laboratory for Scientific Computing (LNCC), Brazil. He has productivity in research scholarship, awarded by Brazil's National Council for Technological and Scientific Development (CNPq). His research interests include sensor networks, the IoT, network characterization, modeling, and analysis, and network science.

**Heder Soares Bernardino** received a D.Sc. degree in Computational Modeling from National Laboratory for Scientific Computing (2012) and is currently a Professor at Federal University of Juiz de Fora, Brazil. He has a Research Productivity Grant from the National Council for Scientific and Technological Development (CNPq). Heder has experience in Computer Science, and his research interests are evolutionary computation, data mining, and high-performance computing.